Step 2 — The Style Reviewer, Grounded in Your Style Guide With RAG — Hints

Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and what proves retrieval works

Hint 1 — The idea, and the exact pieces

The Style reviewer judges code against your **real** style guide, not general knowledge — so it has to *search* that guide. That's the RAG from Doc08/Project 3, reused:

- `data/style_guide.md` — your real rules, one per paragraph. Put at least one **specific, unusual** rule in it (something a model wouldn't invent from training) — that's what proves retrieval is doing the work.
- `src/rag/store.py` — `get_collection()` (a fresh Chroma collection).
- `src/rag/ingest.py` — `build_style_store(collection)` chunks the guide into rules and embeds them.
- `src/rag/retriever.py` — `retrieve_rules(collection, query)` returns the top-k matching rules.
- `src/agents/style_reviewer.py` — `run_style(model, collection, files)`.

Hint 2 — The plan, and what proves retrieval works

```
run_style(model, collection, files):
    changed = all added lines across files
    if not changed: return []
    rules = retrieve_rules(collection, "\n".join(changed))   # from YOUR guide
    result = (prompt | model.with_structured_output(StyleFindings)).invoke(
        {"rules": rules, "lines": changed})
    return [Finding(low, "style", <cited rule>) for each violation]
```

The prompt must say **only use the rules provided — never invent a rule**. The rules passed in came from retrieval, so a finding can only cite a rule that is actually in your guide.

What proves retrieval works: a `camelCase` function name in the diff should retrieve *your* snake_case rule (S-9) and the finding should cite it — a result the model couldn't produce from general knowledge, because "no camelCase, even for ported JS" is your rule, not a universal one. To unit-test retrieval with no model, use a small offline L2-normalized keyword embedding (like Doc08/Project 3). The whole file set is in the Solution.